

Policies & @can Directive

In the last lesson, we added authorization to the edit and delete buttons on the job listings. We can still do the actual update and delete without owning the listing, which we need to change, but first I want to show you how to create a new policy and use the @can directive.

Open the terminal and run the following command to create a new policy:

```
php artisan make:policy JobPolicy --model=Job
```

This will create a file at `app/Policies/JobPolicy.php`. Open this file and you will see methods like `viewAny`, `view`, `create`, `update`, `delete`, etc. This is where you can define the authorization rules for the `Job` model.

let's add the following code to the `update` method:

```
public function update(User $user, Job $job)
{
    return $user->id === $job->user_id;
}
```

This method will return `true` if the user ID of the currently authenticated user is the same as the user ID of the job listing. If it returns `true`, the user can update the job listing. If it returns `false`, the user cannot update the job listing.

Let's add the following code to the `delete` method:

```
public function delete(User $user, Job $job)
{
    return $user->id === $job->user_id;
}
```

This will do the same as the `update` method, but for the `delete` method.

Register the Policy

We need to register this policy within an auth service provider. Let's create a new auth service provider by running the following command:

```
php artisan make:provider AuthServiceProvider
```

A service provider is a class that registers bindings in the service container. It contains a `boot` method that is called when the application is booted. This is where we can register our policies.

Open the file `app/Providers/AuthServiceProvider.php` and add the following imports:

```
use App\Models\Job;
use App\Policies\JobPolicy;
use Illuminate\Foundation\Support\Providers\AuthServiceProvider as
ServiceProvider;
```

Remove this import:

```
use Illuminate\Support\ServiceProvider;
```

This may be a bit confusing, so let me explain why we switched these imports. When you generate a service provider in Laravel the default class that gets extended is that `Illuminate\Support\ServiceProvider`. This is because service providers are typically meant to be used to register services, bindings, etc within the service container, which is the primary purpose of the `ServiceProvider` class.

However, for certain types of service providers, like `AuthServiceProvider`, Laravel provides specialized base classes such as this `Illuminate\Foundation\Support\Providers\AuthServiceProvider` that include additional functionality specific to that domain, such as the `registerPolicies` method, which is what we need to call. That method is not available in the original `Illuminate\Support\ServiceProvider` class.

Add the following property above the `register` method:

```
protected $policies = [
    Job::class => JobPolicy::class,
];
```

The `$policies` array is used to define which policy class should be used for a specific Eloquent model. In this case, it maps the `Job` model (`Job::class`) to the `JobPolicy` class (`JobPolicy::class`).

Then, add the following code to the `boot` method:

```
public function boot()
{
    $this->registerPolicies();
}
```

This will register the policies.

Use the `@can` Directive

Now that we have a policy, we can use the `@can` directive to check if the user can update or delete the job listing. Open the file `resources/views/jobs/show.blade.php` and change the edit and delete buttons to

the following:

```
@can('update', $job)
<div class="flex space-x-3 ml-4">
  <a
    href="{{ route('jobs.edit', $job->id) }}"
    class="px-4 py-2 bg-blue-500 hover:bg-blue-600 text-white rounded"
  >Edit</a>
  <!-- Delete Form -->
  <form
    method="POST"
    action="{{ route('jobs.destroy', $job->id) }}"
    onsubmit="return confirm('Are you sure you want to delete this job?');"
  >
    @csrf @method('DELETE')
    <button
      type="submit"
      class="px-4 py-2 bg-red-500 hover:bg-red-600 text-white rounded"
    >
      Delete
    </button>
  </form>
  <!-- End Delete Form -->
</div>
@endcan
```

This is a lot cleaner than the previous code. We can use the `@can` directive to check if the user can update or delete the job listing.

We still need to stop the actual update and delete without owning the listing, which we need to change. We will do this in the next lesson.